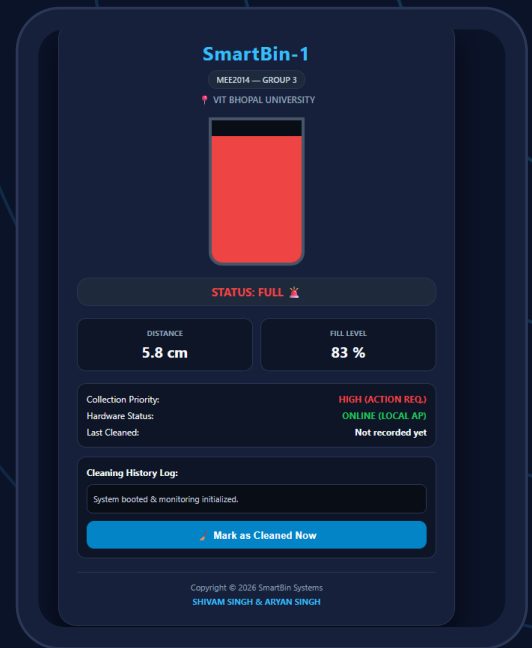


SmartBin-1

Automated IoT Waste-Level Monitoring & Alert System

A standalone ESP32 smart-bin node that measures fill level with ultrasonic ranging, signals it with LEDs and a buzzer, and streams live telemetry to any phone over its own Wi-Fi hotspot — no internet required.



Live dashboard captured from the working prototype

ESP32

Wi-Fi AP host

HC-SR04

ranging sensor

3-level

LED + buzzer alerts

±2.88 %

worst-case error

SUBMITTED BY

Shivam Singh

Reg. No. 25BCE10736

shivam.25bce10736@vitbhupal.ac.in

Aryan Singh

Reg. No. 25BCE10798

aryan.25bce10798@vitbhupal.ac.in

VIT Bhopal University

School of Computer Science and Engineering (SCOPE)
Kotri Kalan, Ashta, Dist. Sehore – 466114, Madhya Pradesh, India

Prototype Unit V1.0

Smart Infrastructure · Urban Sanitation

Contents

- 1. Abstract & Project Idea 3
- 2. Problem Statement & Objectives 3
- 3. System Architecture 3
- 4. Bill of Materials 4
- 5. Hardware Design & Connections 5
- 6. Working Principle & Mathematics 6
- 7. Firmware Design 7
- 8. Web Dashboard 7
- 9. Setup & Deployment Guide 8
- 10. Results & Testing 9
- 11. Error Margin & Limitations 9
- 12. Business Model & Future Scope 10
- 13. Conclusion 11
- 14. Appendix A — Firmware Source 12

14.4 cm calibrated bin depth	200 ms sensing cycle	500 ms dashboard refresh	3 alert levels	0 internet needed
--	--	--	--------------------------------------	---

01 Abstract & Project Idea

Municipal and institutional waste collection still relies largely on fixed schedules and manual inspection. Bins are emptied when they are already overflowing or when they are still nearly empty, which wastes fuel and labour and leaves sanitation problems unaddressed. **SmartBin-1** is a low-cost IoT monitoring node designed to fix this for campus environments.

The prototype combines an **HC-SR04 ultrasonic sensor** and an **ESP32 microcontroller** to measure how far the waste surface is from the lid, converts that distance into a volumetric fill percentage, and reports it three ways: **status LEDs** (green, yellow, red), an **active piezo buzzer** that sounds at critical fill, and an **embedded web dashboard** served by the ESP32 itself. The board runs as its own Wi-Fi Access Point, so any phone or laptop can connect directly and view live telemetry with no router, cloud service or internet connection.

Project at a glance

Unit: SmartBin-1 (Prototype V1.0) · **Domain:** Smart infrastructure, urban sanitation, real-time telemetry
Course / Group: MEE2014, Group 3 · **Board:** ESP32 DevKit V1 · **Sensing:** HC-SR04 ultrasonic ranging

02 Problem Statement & Objectives

Fixed-schedule collection creates three recurring problems: overflowing bins between rounds, wasted trips to bins that are barely used, and no data to plan better routes. A per-bin sensor that reports its own fill state turns collection from schedule-driven into demand-driven.

Objectives

- Measure container fill level continuously and without physical contact.
- Convert raw ultrasonic distance into a calibrated fill percentage (0 – 100 %).
- Give an immediate on-site signal: green (OK), yellow (half full) and red plus buzzer (full).
- Expose live readings through a responsive web dashboard that works entirely offline over a local Wi-Fi hotspot.
- Keep hardware cost low enough to be deployable across many bins.

03 System Architecture

The ESP32 is the single controller. It triggers the sensor, computes distance and fill level, drives the local indicators, and simultaneously hosts a Wi-Fi Access Point and HTTP server. A connected client polls a small JSON endpoint twice a second and redraws the dashboard.

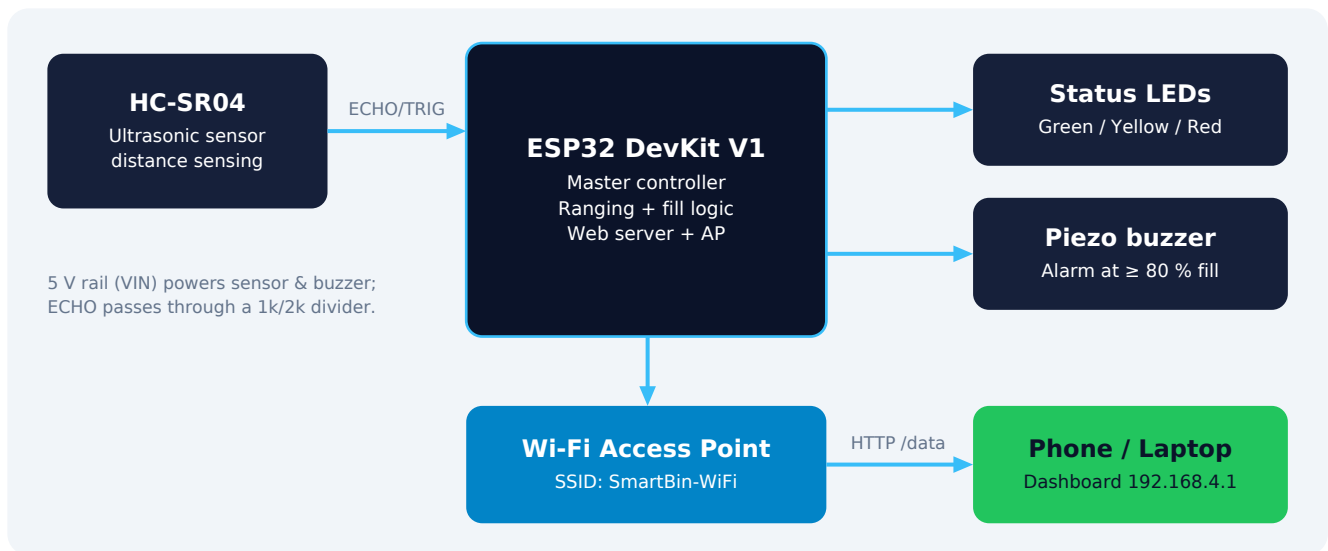


Figure 1 — SmartBin-1 system architecture and signal flow.

Layer	Element	Role
Sensing	HC-SR04 ultrasonic module	Emits a 40 kHz burst and returns an echo pulse whose width encodes distance
Processing	ESP32 (ESP-WROOM-32)	Ranging, fill calculation, threshold logic, Wi-Fi AP, HTTP server
Local output	3 LEDs + active buzzer	On-bin status indication and audible alarm at $\geq 80\%$ fill
Remote output	Web dashboard at 192.168.4.1	Live distance, fill level, priority, cleaning log

04 Bill of Materials

Component	Specification	Qty	Primary function
ESP32 DevKit V1	30-pin, ESP-WROOM-32	1	Master controller and Access Point host
HC-SR04	Ultrasonic distance sensor, 5 V logic	1	Non-contact distance measurement
Resistor 1 kΩ	1/4 W	3	R1 of the divider, plus two in series forming R2 = 2 k Ω
LED — Green / Yellow / Red	5 mm standard	3	Fill < 50 % / 50 – 79 % / $\geq 80\%$
Resistor 220 Ω	1/4 W	3	Current limiting for each LED
Active piezo buzzer	5 V active sound generator	1	Acoustic alarm on critical fill
Solderless breadboard	830-point, dual bus rails	1	Prototyping base
Jumper wires	M-M and M-F	15+	Interconnects
Micro-USB cable	Data-capable	1	Programming and 5 V power

Note on the resistor count

The divider needs a 1 k Ω resistor (R1) and a 2 k Ω ground leg (R2), built from two 1 k Ω resistors in series — three 1 k Ω resistors in total.

05 Hardware Design & Connections

5.1 Level shifting the ECHO signal

The HC-SR04 drives its ECHO pin at 5 V, while ESP32 GPIO pins are only rated for 3.3 V. Feeding ECHO straight into a GPIO stresses the input and can damage the chip over time. A resistive divider (1 kΩ over 2 kΩ) scales the pulse to about 3.33 V, which is safe for GPIO18.

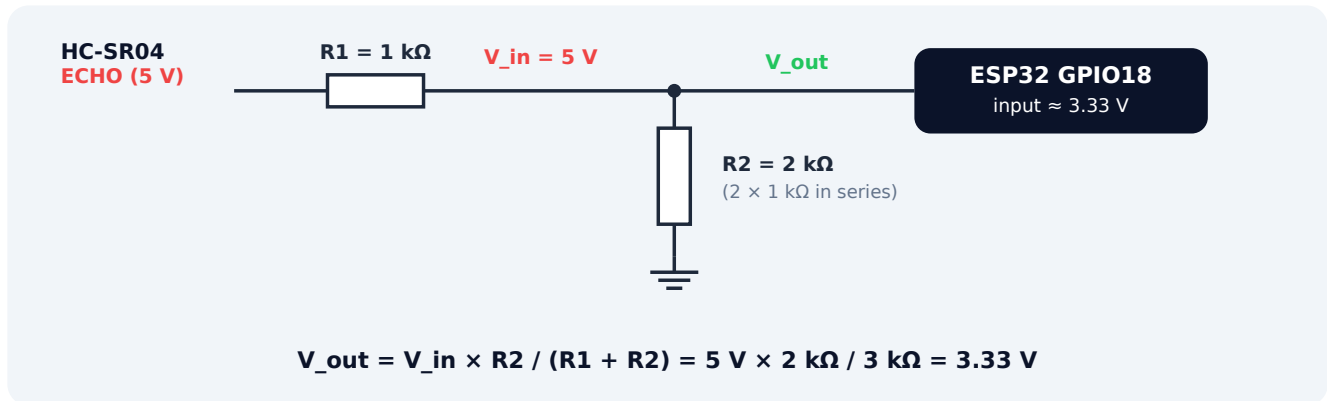


Figure 2 — ECHO voltage divider protecting the ESP32 input.

5.2 Wiring diagram

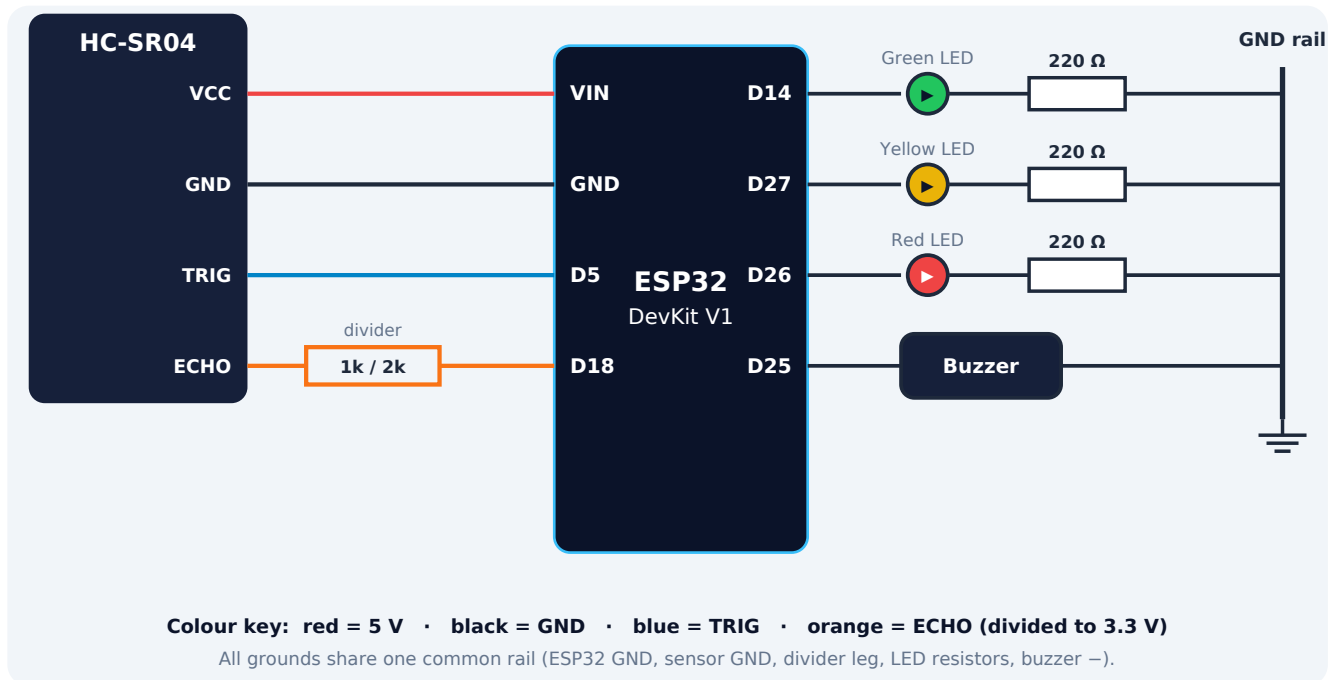


Figure 3 — Complete wiring of SmartBin-1 (breadboard prototype).

5.3 Pin interconnect table

Source	Terminal	Destination	Function
ESP32	VIN (5 V)	Red power rail (+)	5 V master bus
ESP32	GND	Blue ground rail (–)	Common ground
HC-SR04	VCC	Red power rail (+)	Sensor power
HC-SR04	GND	Blue ground rail (–)	Sensor ground

Source	Terminal	Destination	Function
HC-SR04	TRIG	GPIO5 (D5)	Trigger pulse output
HC-SR04	ECHO	1 kΩ (R1) → node	5 V echo signal in
Divider	Mid-node	GPIO18 (D18)	3.33 V echo signal
Divider	2 kΩ ground leg	Blue ground rail (–)	Reference ground
Green LED	Anode (+)	GPIO14 (D14)	Output control
Yellow LED	Anode (+)	GPIO27 (D27)	Output control
Red LED	Anode (+)	GPIO26 (D26)	Output control
Each LED	Cathode (–)	220 Ω → ground rail	Current limit and return
Active buzzer	Positive (+)	GPIO25 (D25)	Alarm driver
Active buzzer	Negative (–)	Blue ground rail (–)	Return path

06 Working Principle & Mathematics

6.1 Ultrasonic ranging

A 10 μs pulse on TRIG makes the HC-SR04 emit an 8-cycle burst at 40 kHz. The module raises ECHO for as long as the sound takes to travel to the waste surface and back. With the speed of sound $v = 343 \text{ m/s} = 0.0343 \text{ cm/μs}$, and echo time t in microseconds:

$$\text{Distance (cm)} = t \times 0.0343 / 2$$

6.2 Volumetric calibration

The bin was calibrated at an empty depth $D_{\text{empty}} = 14.4 \text{ cm}$ (sensor to bottom). The bin is treated as full when the waste surface reaches $D_{\text{full}} = 4.0 \text{ cm}$ from the sensor, leaving headroom below the sensor's minimum reliable range.

$$\text{Fill \%} = (D_{\text{empty}} - D_{\text{measured}}) / (D_{\text{empty}} - D_{\text{full}}) \times 100$$

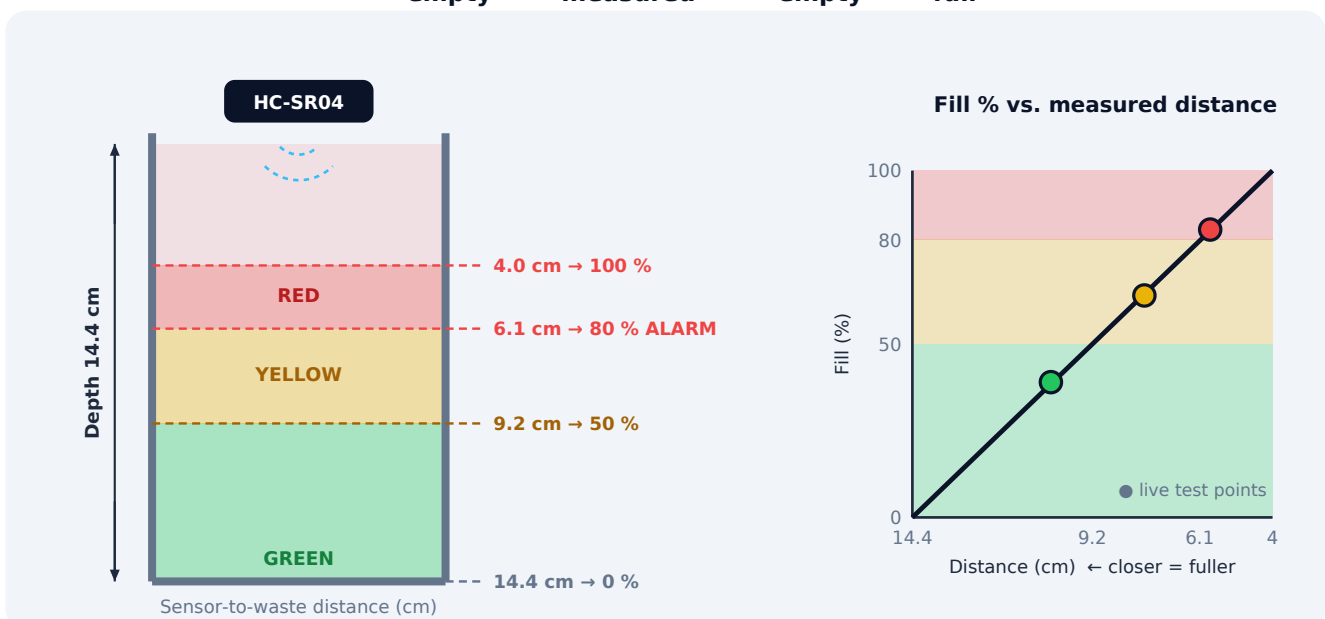


Figure 4 — Bin geometry with threshold levels (left) and the distance-to-fill mapping with the three live test points (right).

6.3 State output matrix

Because fill is linear in distance over a 10.4 cm span, the 50 % and 80 % alert points fall at 9.2 cm and 6.08 cm respectively ($14.4 - 0.5 \times 10.4$ and $14.4 - 0.8 \times 10.4$).

Distance D	Fill level	LED	Buzzer	Dashboard status
D > 9.2 cm	< 50 %	GPIO14 (Green)	OFF	STATUS: OK
6.08 cm < D ≤ 9.2 cm	50 - 79 %	GPIO27 (Yellow)	OFF	STATUS: HALF FULL
D ≤ 6.08 cm	≥ 80 %	GPIO26 (Red)	ON (2500 Hz)	STATUS: FULL

07 Firmware Design

The firmware (SmartBin_Local.ino) is written for the Arduino-ESP32 core using the built-in WiFi and WebServer libraries. `setup()` initialises the pins, runs a 1.2 s LED test and starts the Access Point. `loop()` services web clients, takes one distance reading, updates the fill percentage and drives the indicators.

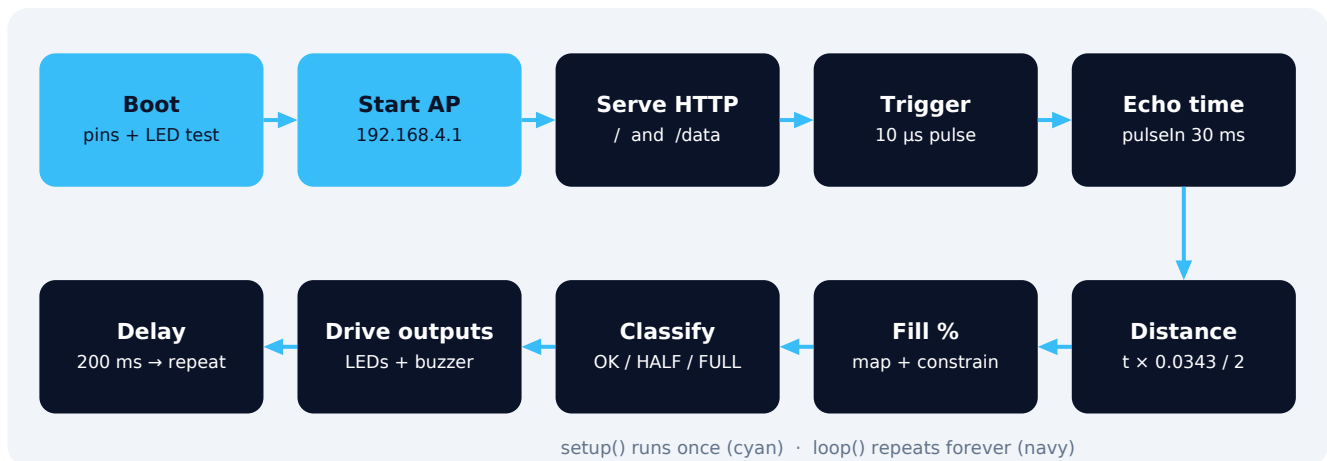


Figure 5 — Firmware execution flow.

Feature	Implementation detail
Access Point	SSID SmartBin-WiFi, WPA2 password set in firmware, fixed address 192.168.4.1
Timeout handling	<code>pulseIn(ECHO, HIGH, 30000)</code> : a lost echo is treated as an empty bin (14.4 cm, 0 %)
Range clamping	Distance clamped to 2.0 - 14.4 cm; fill constrained to 0 - 100 %
Alert logic	≥ 80 % → red LED + 2500 Hz tone; ≥ 50 % → yellow; otherwise green
Endpoints	/ returns the embedded HTML dashboard; /data returns JSON {distance, fill, status}
Storage	Dashboard is held in flash (PROGMEM) as a raw string literal — no filesystem needed

08 Web Dashboard

The dashboard is a single self-contained page with a dark, card-based layout that adapts to phones and laptops. JavaScript polls `/data` every 500 ms and updates the bin graphic, colour-coded status pill, metric tiles and collection priority.

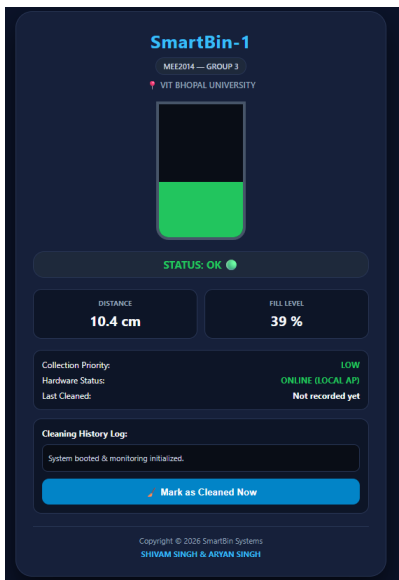


Figure 6 — Dashboard in the OK state (distance 10.4 cm, fill 39 %).

- **Animated bin graphic** whose fill height and colour follow the live reading.
- **Status pill** — OK (green), HALF FULL (yellow), FULL (red).
- **Metric tiles** for distance (cm) and fill level (%).
- **Collection priority** — LOW, MEDIUM or HIGH (action required).
- **Hardware status** line confirming the local AP link.
- **Cleaning log** with a “Mark as Cleaned Now” button and automatic detection when a full bin is emptied.

09 Setup & Deployment Guide

9.1 Driver and COM port

- Connect the ESP32 with a data-capable micro-USB cable.
- Open Device Manager (Win + X) and expand **Ports (COM & LPT)**.
- A CP210x or CH340 entry with a COM number means the driver is present. A yellow warning or an entry under *Other devices* means the driver is missing: install the Silicon Labs CP210x or CH341SER driver and reboot.

9.2 Arduino IDE and board package

- In **File → Preferences**, add the Espressif board index URL to *Additional Boards Manager URLs*:
https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json
- Open **Tools → Board → Boards Manager**, search *esp32* and install (v2.x or v3.x).
- Select **ESP32 Dev Module** as the board and choose the detected COM port under **Tools → Port**.

9.3 Compile and flash

- Paste the firmware from Appendix A into a new sketch and save it as `SmartBin_Local.ino`.
- Press **Verify**, then **Upload**. If the console shows `Connecting...__`, hold the BOOT button until writing starts.
- Open the Serial Monitor at **115200 baud** and press EN/RST to see the start-up banner.

9.4 Connecting to the dashboard

Method	Steps
A — Local hotspot (recommended)	Join Wi-Fi SmartBin-WiFi (password 12345678). Turn off mobile data so the phone does not route around the offline network. Open <code>http://192.168.4.1</code> in any modern browser.
B — USB serial	Keep the board connected to a laptop and open the port at 115200 baud in Arduino IDE or VS Code to read diagnostic output.

10 Results & Testing

The prototype was tested by placing material at three different heights in the calibrated container and reading the dashboard. The three captures below show each alert state, and the table compares the fill percentage calculated from the displayed distance with the value the device reported.

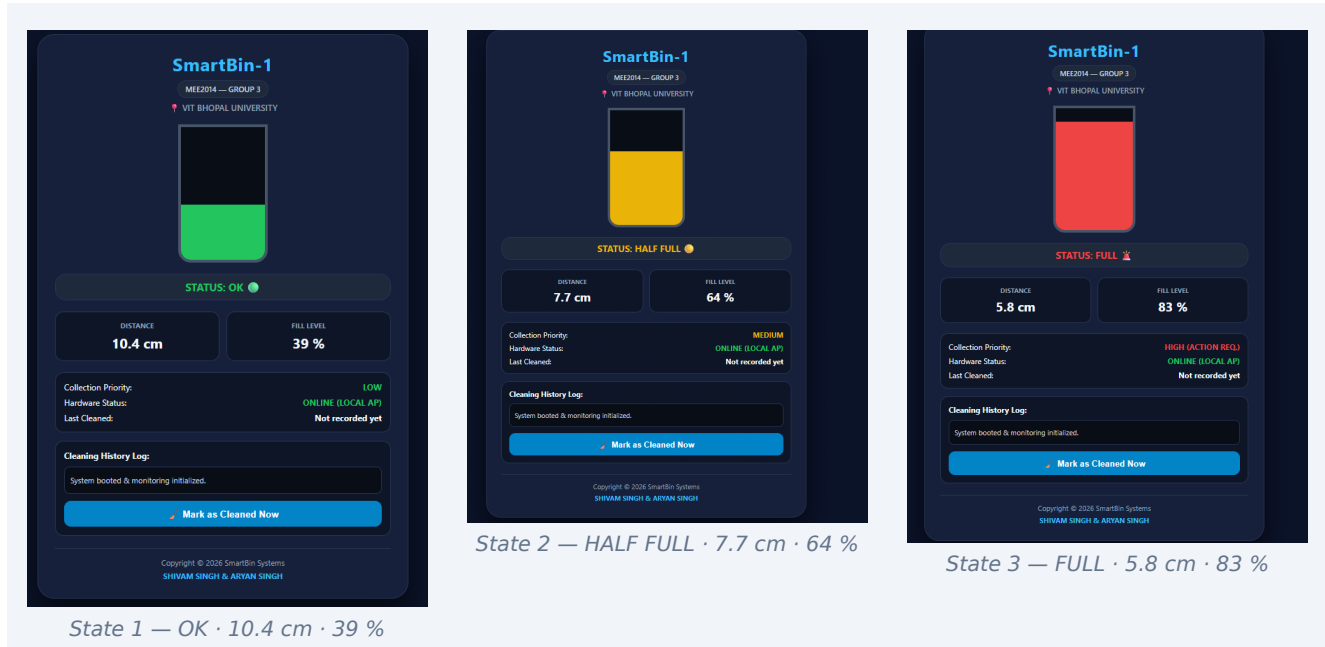


Figure 7 — Live dashboard in the three alert states.

Test	Displayed distance	Calculated fill	Reported fill	State / priority	Indicators
1	10.4 cm	38.5 %	39 %	OK · LOW	Green LED, buzzer off
2	7.7 cm	64.4 %	64 %	HALF FULL · MEDIUM	Yellow LED, buzzer off
3	5.8 cm	82.7 %	83 %	FULL · HIGH (action req.)	Red LED, buzzer on

The reported values match the formula to within rounding: the dashboard shows distance to one decimal place, while the device computes fill from the unrounded reading. Each state produced the expected colour, priority label and hardware response.

11 Error Margin & Limitations

Accuracy

The HC-SR04 has a nominal ranging accuracy of ± 0.3 cm under ideal line-of-sight conditions. Over the 10.4 cm working span this gives:

$$\text{Error} = \pm (0.3 / 10.4) \times 100 \% \approx \pm 2.88 \%$$

Known limitations

- **Surface scattering.** Soft materials (paper, textile) absorb ultrasound and steeply angled surfaces reflect it away, which can cause a lost echo. The firmware then reports an empty bin, so a missed echo can under-report fill.

- **Single-point measurement.** The sensor reads the nearest point below it, so uneven piles under the sensor can misrepresent the true volume.
- **Wi-Fi range and clients.** The on-board antenna in Access Point mode reaches roughly 15 – 30 m line of sight, and the ESP32 hotspot serves only a few clients at once by default.
- **Browser-side log.** The cleaning history and “Last cleaned” time live in the viewer's browser, so they reset when the page is reloaded and are not stored on the device.
- **Prototype scale.** One node, no cloud storage and no long-term data logging yet.

12 Business Model & Future Scope

SmartBin can be commercialised as hardware plus a subscription platform: the node sells as a low-cost device (author-estimated unit cost about USD 12, retail USD 35 – 45), while a central dashboard is offered as a per-bin SaaS subscription. Target customers are university campuses, municipal corporations, airports and commercial malls.

12.1 Deployment ecosystem

Because each node is self-contained and needs only a 5 V supply and (optionally) its own Wi-Fi hotspot, the same design deploys across very different sites with no change to the hardware. Four sector groups cover the highest-impact locations:

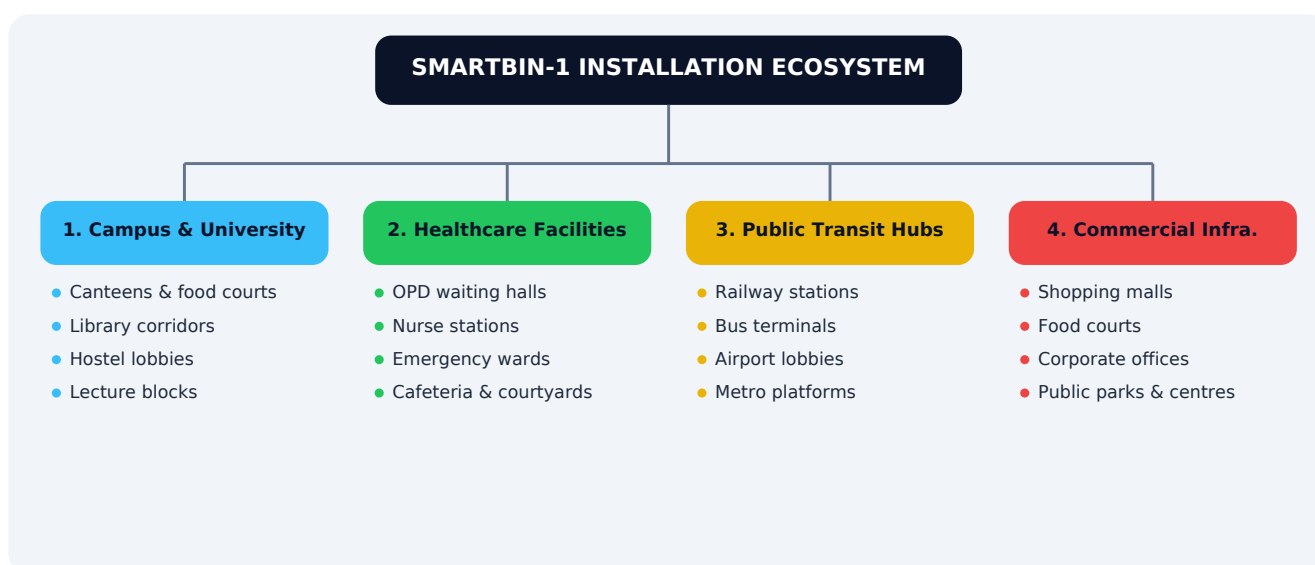


Figure 8 — SmartBin-1 installation ecosystem across four deployment sectors.

Sector	Locations	Why SmartBin-1 helps
University & educational campuses	Canteens & food courts · library corridors & lobbies · hostel common areas	Handles sudden volume spikes at peak lunch hours; flags overflow in high-footfall areas that affect campus aesthetics; tracks high-volume packaging and dry-waste generation in hostels.
Hospitals & healthcare facilities	OPD waiting halls & corridors · nurse stations & wards · cafeterias & emergency wards	Prevents public-area clutter and cross-contamination risk; lets housekeeping staff check fill levels without disturbing patients; timestamped cleaning logs support healthcare compliance audits.
Public transit hubs	Railway platforms · airport lobbies · bus terminals · metro platforms	Enables remote, real-time monitoring across high-density transit zones where manual checks are impractical.

Sector	Locations	Why SmartBin-1 helps
Commercial & urban infrastructure	Shopping malls · food courts · corporate centres · public parks	Replaces rigid fixed-time cleaning schedules with dynamic, priority-based route planning.

12.2 Technology roadmap

Roadmap step	Upgrade	Benefit
1 Wide-area connectivity	Replace local AP with LoRaWAN or NB-IoT (LTE-M)	Multi-kilometre range and telemetry to a central municipal cloud
2 Solar harvesting	5 V mini solar panel, 3.7 V 18650 Li-ion cell, deep-sleep firmware	Continuous outdoor operation with no wired power
3 Route optimisation	Aggregate fill data from hundreds of nodes	Automatic truck routes ranked by real-time priority

13 Conclusion

SmartBin-1 demonstrates that a compact, low-cost ESP32 node can turn an ordinary bin into a self-reporting one. The prototype measures fill level with a worst-case error near $\pm 2.88\%$, signals status locally with LEDs and a buzzer, and delivers live data to any phone through its own offline Wi-Fi hotspot. Testing confirmed correct behaviour in all three alert states. With longer-range connectivity, solar power and a route-planning back end, the same design scales from a campus pilot to a city-wide demand-driven collection system.

14 Appendix A — Firmware Source

Core firmware of SmartBin_Local.ino. The embedded HTML/CSS/JavaScript dashboard (HTML_DASHBOARD) is stored in the same sketch and is omitted here for length; status emoji in the strings are also omitted from this printout.

```
#include <WiFi.h>
#include <WebServer.h>

const char* AP_SSID = "SmartBin-WiFi";
const char* AP_PASS = "12345678";

// Hardware pin mappings
const int TRIG_PIN      = 5;    // GPIO5   / D5
const int ECHO_PIN      = 18;   // GPIO18 / D18 (via voltage divider)
const int GREEN_LED_PIN = 14;   // GPIO14 / D14
const int YELLOW_LED_PIN = 27;  // GPIO27 / D27
const int RED_LED_PIN   = 26;   // GPIO26 / D26
const int BUZZER_PIN    = 25;   // GPIO25 / D25

// Container configuration (calibrated 14.4 cm depth)
const float BIN_DEPTH_CM = 14.4; // empty bin
const float FULL_THRESHOLD_CM = 4.0; // distance at 100 % fill

float distance_cm = 14.4;
int fill_percentage = 0;
String status_msg = "OK";

WebServer server(80);

const char HTML_DASHBOARD[] PROGMEM = R"rawliteral(
... embedded dashboard page (HTML + CSS + JavaScript) ...
)rawliteral";

void handleRoot() {
  server.send(200, "text/html", HTML_DASHBOARD);
}

void handleData() {
  String json = "{";
  json += "\"distance\": " + String(distance_cm, 1) + ",";
  json += "\"fill\": " + String(fill_percentage) + ",";
  json += "\"status\": \"" + status_msg + "\"";
  json += "}";
  server.send(200, "application/json", json);
}

void setup() {
  Serial.begin(115200);
  delay(1000);

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(GREEN_LED_PIN, OUTPUT);
  pinMode(YELLOW_LED_PIN, OUTPUT);
  pinMode(RED_LED_PIN, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);

  // Hardware boot test
  digitalWrite(GREEN_LED_PIN, HIGH);
  digitalWrite(YELLOW_LED_PIN, HIGH);
  digitalWrite(RED_LED_PIN, HIGH);
  delay(1200);
  digitalWrite(GREEN_LED_PIN, LOW);
  digitalWrite(YELLOW_LED_PIN, LOW);
  digitalWrite(RED_LED_PIN, LOW);
  noTone(BUZZER_PIN);

  // Standalone Access Point
  WiFi.mode(WIFI_AP);
  WiFi.softAP(AP_SSID, AP_PASS);

  Serial.println("\n=====");
  Serial.println(" SmartBin Standalone Hotspot Active ");
  Serial.println(" Connect Phone Wi-Fi to: SmartBin-WiFi");
  Serial.println(" Password: " + String(AP_PASS));
  Serial.print(" Open Browser URL: http://");
  Serial.println(WiFi.softAPIP()); // 192.168.4.1
  Serial.println("=====");

  server.on("/", handleRoot);
  server.on("/data", handleData);
  server.begin();
}
```

```

void loop() {
  server.handleClient();

  // 1. Ultrasonic distance measurement
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  long duration = pulseIn(ECHO_PIN, HIGH, 30000);

  if (duration == 0) {
    distance_cm = BIN_DEPTH_CM;
    fill_percentage = 0;
  } else {
    distance_cm = duration * 0.0343 / 2.0;

    if (distance_cm > BIN_DEPTH_CM) distance_cm = BIN_DEPTH_CM;
    if (distance_cm < 2.0) distance_cm = 2.0;

    fill_percentage = round(((BIN_DEPTH_CM - distance_cm) /
                           (BIN_DEPTH_CM - FULL_THRESHOLD_CM)) * 100.0);
    fill_percentage = constrain(fill_percentage, 0, 100);
  }

  // 2. Hardware alert state logic
  if (fill_percentage >= 80) {
    digitalWrite(GREEN_LED_PIN, LOW);
    digitalWrite(YELLOW_LED_PIN, LOW);
    digitalWrite(RED_LED_PIN, HIGH);
    tone(BUZZER_PIN, 2500);
    status_msg = "FULL";
  } else if (fill_percentage >= 50) {
    digitalWrite(GREEN_LED_PIN, LOW);
    digitalWrite(YELLOW_LED_PIN, HIGH);
    digitalWrite(RED_LED_PIN, LOW);
    noTone(BUZZER_PIN);
    status_msg = "HALF FULL";
  } else {
    digitalWrite(GREEN_LED_PIN, HIGH);
    digitalWrite(YELLOW_LED_PIN, LOW);
    digitalWrite(RED_LED_PIN, LOW);
    noTone(BUZZER_PIN);
    status_msg = "OK";
  }

  delay(200);
}

```